

Análisis de Algoritmos I

Algoritmos de búsqueda

L.C.C. María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Algoritmos de búsqueda

- Distintos algoritmos pueden ser utilizados para resolver un mismo problema, y la eficiencia puede ser muy diferente entre un algoritmo y otro.
- **Objetivo:** saber si un dato se encuentra o no dentro de una colección, así como la posición que ocupa.
- Desarrollando intuición: [Juguemos al juego de adivinar](#) 😊

Búsqueda lineal (Linear-search)

- Localiza un valor concreto dentro de una lista comprobando cada elemento **uno a uno**.
- Muy **sencillo e intuitivo**.
- **No** requiere que los datos estén **ordenados**.
- Ideal cuando se trabaja con **conjuntos pequeños** de datos.

Linear Search

Find '20'

0	1	2	3	4	5	6	7	8
10	15	30	70	80	60	20	90	40

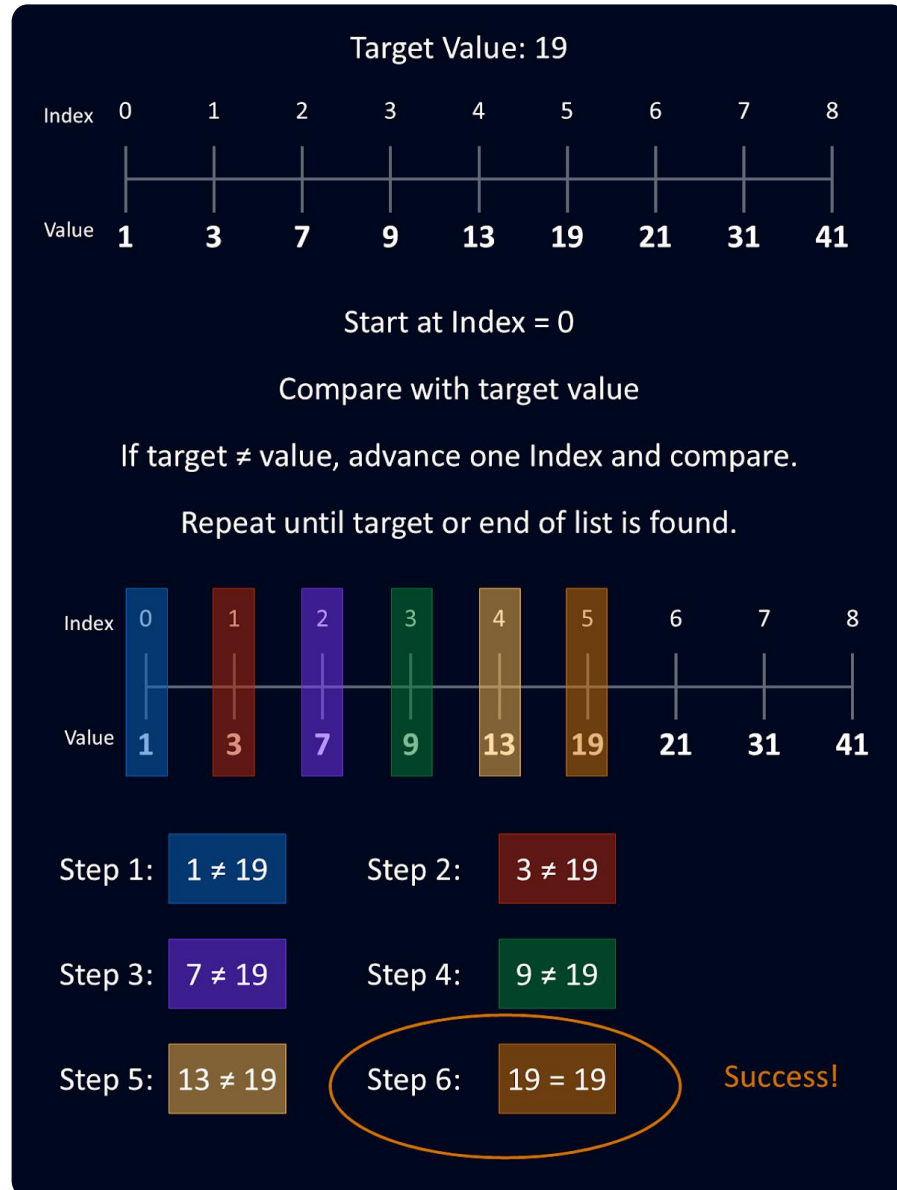
Búsqueda lineal: pseudocódigo

Algoritmo 1

Input: Un arreglo $a_1, a_2, \dots, a_n$, para $n > 0$ y
 X (el valor objetivo)

Output: i si $a_i = X$ y
-1 si no existe ninguna i tal que $a_i = X$.

```
for i = 1 to n do
    if array[i] == X then return i y parar
return -1
```



Ejemplo
Búsqueda
lineal

Búsqueda lineal: Análisis

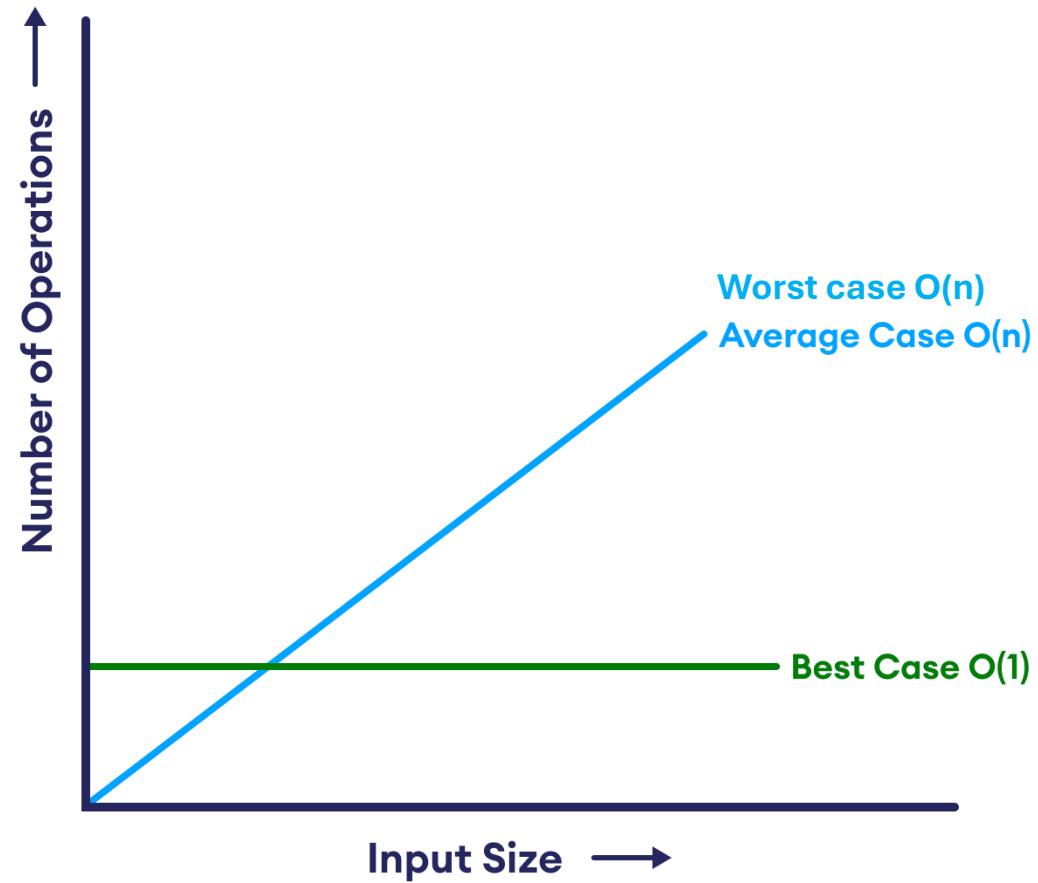
- **Mejor caso:** el valor objetivo es el **primero** en la lista.
- Número de comparaciones: 1
- Complejidad temporal: $O(1)$

- **Peor caso:** el valor objetivo es el **último** de la lista ó **no se encuentra** en la lista.
- Número de comparaciones: n
- Complejidad temporal: $O(n)$

Búsqueda lineal: Análisis

- **Caso promedio:** el valor objetivo está **en la mitad** de la lista.
- Número de comparaciones: $n/2$
- Complejidad temporal: $O(n)$
- El análisis del caso promedio no es fácil de realizar en la mayoría de los casos prácticos y rara vez se realiza. En el análisis del caso promedio, debemos considerar cada entrada, su frecuencia y el tiempo que tarda, lo cual puede no ser posible en muchos escenarios.

Búsqueda lineal: Resumen



Búsqueda binaria iterativa (Binary-search)

- Es un algoritmo diseñado para encontrar de forma eficiente un valor en un **conjunto de datos ordenado** (ascendente o descendente). Este es un requisito obligatorio.
- Pertenece a la estrategia de diseño ***Divide y Vencerás***, ya que no revisa cada elemento uno a uno, sino que **reduce el intervalo de búsqueda a la mitad** con cada paso.
- Es mucho más rápida que la búsqueda lineal, sobre todo cuando se trata de **grandes conjuntos de datos**.

Binary Seach



	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0	1	2	3	M=4	5	6	7	8	H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5	6	M=7	8	H=9
23 < 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5.M=5	H=6	7	8	9
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91

Búsqueda binaria: pseudocódigo

Algoritmo 2

Input: Un arreglo ordenado $a_1, a_2, \dots, a_n$, para $n > 0$ y X (el valor objetivo), donde $a_1 < a_2 < \dots < a_n$.

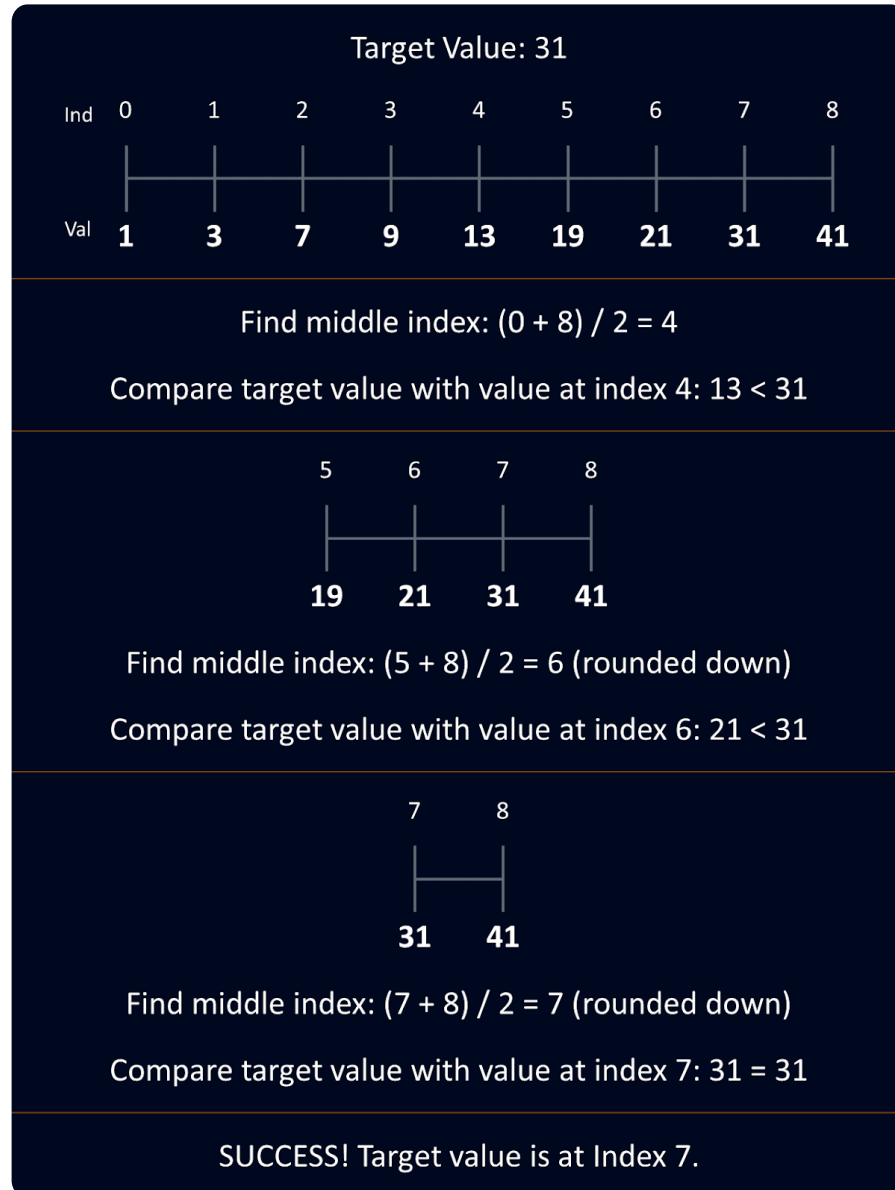
Output: $medio$ si $a_{medio} = X$ y -1 si no se encuentra en la lista.

```
izq = 0                # índice del primer elemento
der = n                # índice del último elemento

while izq ≤ der do

    medio =  $\frac{izq + der}{2}$     # calculamos el índice medio redondeado hacia abajo
    If  $a_{medio} = X$  then return medio y parar # se encontró, regresa medio
    If  $a_{medio} > X$  then der=medio-1        # recalculamos el índice del último elm
    else izq =medio+1                       # recalculamos el índice del primer elm

return -1                # no se encontró, regresa -1
```



Ejemplo
Búsqueda
binaria

Búsqueda binaria: Análisis

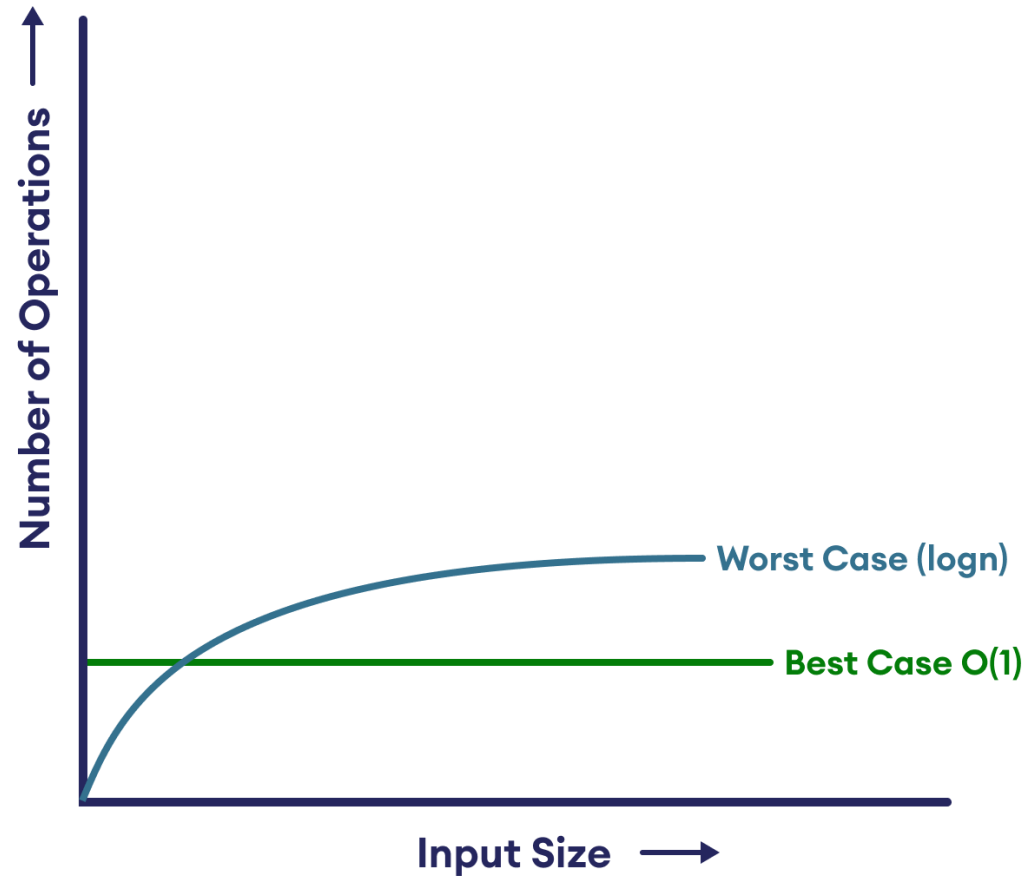
- **Mejor caso:** el valor objetivo es el **elemento central** del arreglo.
- Número de comparaciones: 1
- Complejidad: $O(1)$

- **Peor caso:** el valor objetivo se encuentra **lejos del elemento central** ó **no se encuentra** en la lista.
- Número de comparaciones: $\log_2 n$
- Complejidad: $O(\log n)$

Búsqueda binaria: Análisis

- **Caso promedio:** En promedio, el algoritmo realiza aproximadamente la mitad de las comparaciones necesarias en el peor de los casos
- Número de comparaciones: $(\log n) / 2$
- Complejidad: $O(\log n)$

Búsqueda binaria (iterativa): Resumen



Bibliografía

- Lee, R.C.T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2007). Introducción al diseño y análisis de algoritmos. Un enfoque estratégico. México, D.F. Mc Graw Hill.
- <https://www.datacamp.com/es/tutorial/linear-search-python>
- <https://www.datacamp.com/es/tutorial/binary-search-python>
- <https://es.khanacademy.org/computing/computer-science/algorithms>
- <https://dipeshpatil.github.io/algorithms-visualiser/#/>
- <https://es.khanacademy.org/computing/computer-science/algorithms/recursive-algorithms/a/recursion>